

课程设计报告

自走棋对战系统

C++17 + SFML 2.6.2 + Spine Runtime 3.8

课程名称	计算机程序设计基础 2
课题名称	自走棋对战系统
班 级	_____
学 号	_____
姓 名	_____
指导教师	_____

2026 年 9 月 4 日

摘要

本项目面向《计算机程序设计基础 2》课程设计要求，使用 C++17、SFML 2.6.2 和 Spine Runtime 3.8 实现一个具有图形界面的自走棋对战系统。系统包含主菜单、帮助、四张地图选择、三种分队选择、三种 AI 策略、商店刷新与购买、拖拽部署、合成、出售、复活、提前开战、固定步长战斗、暂停继续、回合结算和结果页面。核心规则集中在 AutoChessCore 静态库中，SFML 界面只把鼠标操作转换为 GameCommand，AI 也只读取 ReadOnlyGameView 快照并提交同一种命令。游戏数值由 UTF-8 配置文件读取，角色显示由 Spine 的 atlas、png 和 skel 资源驱动。当前 Release x64 工程已生成 AutoChessGame.exe、AutoChessCore.lib 和 AutoChessSpine.lib，并将运行所需 DLL 与 data 目录复制到可执行文件旁。项目运用了抽象类、继承、虚函数多态、运算符重载、模板容器、智能指针和文件读写。

关键词：自走棋；C++17；SFML；Spine；面向对象；策略模式；固定步长；CMake

目录

摘 要	2
目 录	2
1 系统需求分析	4
1.1 设计目的与意义	4
1.2 功能需求	4
1.3 非功能需求	4
1.4 输入与输出要求	5
2 系统总体设计	7
2.1 总体架构	7
2.2 关键设计原则	7
2.3 模块依赖与数据流	8
3 系统详细设计	8
3.1 数据文件设计	8
3.2 类层次、抽象类与多态	8
3.3 运算符重载	9
3.4 经济与准备阶段	10
3.5 战斗、技能与结算	10

3.6 AI 设计	10
3.7 界面设计	10
4 系统实现	11
4.1 固定步长主循环	11
4.2 抽象类、继承与单位角色	11
4.3 统一命令	11
4.4 独立发布路径	12
5 系统测试	12
5.1 测试环境与方法	12
5.2 编译与资源复制结果	12
5.3 功能与异常场景验证	13
5.4 测试结论与不足	13
6 运行界面与结果	13
7 系统调试与遇到的困难	19
7.1 从单文件程序到 CMake 多文件项目	19
7.2 SFML 外部库的编译、链接与运行	19
7.3 Spine 动画库与资源文件接入	20
7.4 路径、字体和动画坐标调试	20
8 使用说明与编程体会	20
8.1 发布程序使用步骤	20
8.2 构建与运行步骤	21
8.3 编程体会	21
9 总结与展望	21
附录 A 关键源程序与源程序清单	22
附录 B 完整课程设计任务书与评分表	24

1 系统需求分析

1.1 设计目的与意义

自走棋系统不是只完成一个算法，而是要让地图选择、经济管理、单位部署、自动战斗、AI 决策和图形显示按同一状态机协同工作。本项目以购买、升级、排列和对战为基本流程，将规则计算与界面绘制分开，使同一条规则不会分别散落在按钮、AI 和战斗动画中。课程设计的主要训练目标是把已经学过的 C++ 类、继承、虚函数、容器和文件操作，应用到一个由多个源文件和外部库组成的完整项目中。

项目的个性化设计体现在四点：第一，五类单位、三种分队和四张地图均由文本配置描述；第二，进攻型、防守型和路线型 AI 通过同一策略接口工作；第三，Spine 动画把准备、移动、攻击和死亡状态与核心战斗事件对应；第四，程序优先读取 EXE 旁的 data 目录，便于把整个生成目录复制到其他位置运行。

1.2 功能需求

功能模块	需求	验收边界
启动与菜单	加载配置、字体、SFML DLL 与 Spine 资源；提供开始、帮助和退出。	关键配置缺失时停止进入主循环，动画失败时可使用静态回退。
开局选择	选择 4 张地图之一、3 个玩家分队之一和 3 种 AI 策略之一。	选择按状态机顺序进行，非法阶段命令由 Match 拒绝。
准备与经济	收入、败者补助、6 槽商店、刷新、购买、8 槽备用区、部署、合成、出售和复活。	金币、容量、部署格和单位归属在修改前统一校验。
战斗与动画	单位沿路线移动、索敌、攻击或治疗，到达守卫后结算伤害，并播放 Spine 动作。	核心按 1/60 秒推进；动画只读取战斗结果。
回合与结果	最多 3 回合，结算死亡、存活、到达守卫和超时状态。	守卫归零立即结束；最大回合后比较双方守卫。
暂停与重开	暂停冻结核心推进，继续后恢复；结果页支持再来一局和返回菜单。	暂停期间仍响应窗口事件和界面绘制。
AI	进攻型、防守型、路线型策略完成购买与部署。	只读取快照并提交统一命令，不直接修改 Match。
构建与运行	CMake 生成三个目标，并复制 DLL 与 data 到 EXE 旁。	整个生成目录可作为一个完整运行单元。

1.3 非功能需求

- 可编译：使用 CMake Presets、Ninja、MSVC x64 和 C++17，外部依赖全部放在 third_party 中。
- 可运行：程序创建 2560×1440 窗口，并使用 1280×720 逻辑坐标统一布局 and 缩放。
- 可维护：核心规则不包含 SFML 头文件；界面、动画、AI、经济和战斗模块按目录分开。
- 可移植：构建后自动复制 SFML DLL、MSVC 运行库和完整 data 目录到 AutoChessGame.exe 旁。
- 可诊断：配置或资源缺失时返回明确错误；Spine 资源加载失败时保留静态单位图形作为回退。
- 可扩展：新增地图和数值主要通过 data 文件完成，新增策略通过实现 IAiStrategy 接口完成。

1.4 输入与输出要求

类型	输入/输出	具体内容
用户输入	鼠标	按钮点击、商店购买、拖拽部署或撤回、合成、出售、复活、暂停和继续。
配置输入	UTF-8 文本	game.cfg、units.cfg、factions.cfg 和 map_01.map 至 map_04.map。
资源输入	字体与动画	Noto Sans SC 字体，以及每种单位的 atlas、png、skel 文件。
程序输出	SFML 窗口	菜单、帮助、地图、HUD、商店、单位动画、血条、路线、提示、暂停层和结果页。
诊断输出	标准错误与退出码	配置路径、资源加载错误和启动失败原因。

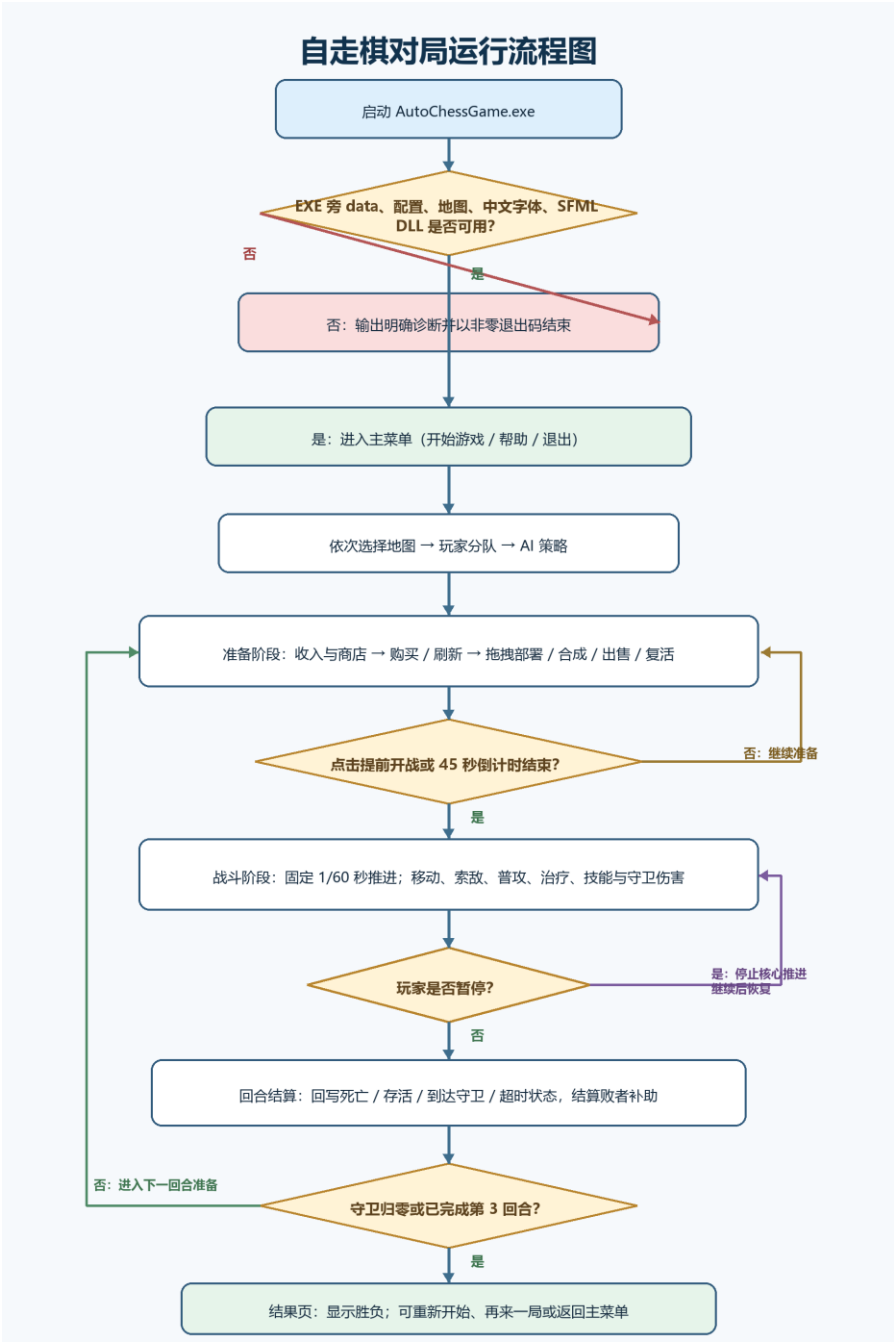


图 1-1 自走棋对局运行流程图

2 系统总体设计

2.1 总体架构

工程由 AutoChessCore、AutoChessSpine 和 AutoChessGame 三个 CMake 目标组成。AutoChessCore 保存配置、模型、经济、战斗、对局和 AI，不依赖 SFML；AutoChessSpine 把项目内的 Spine 3.8 C++ runtime 与 spine-sfml 适配层编译为静态库，并链接 SFML graphics、window、system；AutoChessGame 负责窗口、页面、输入、绘制、动画状态和运行路径，并链接前两个静态库及三个 SFML 导入目标。

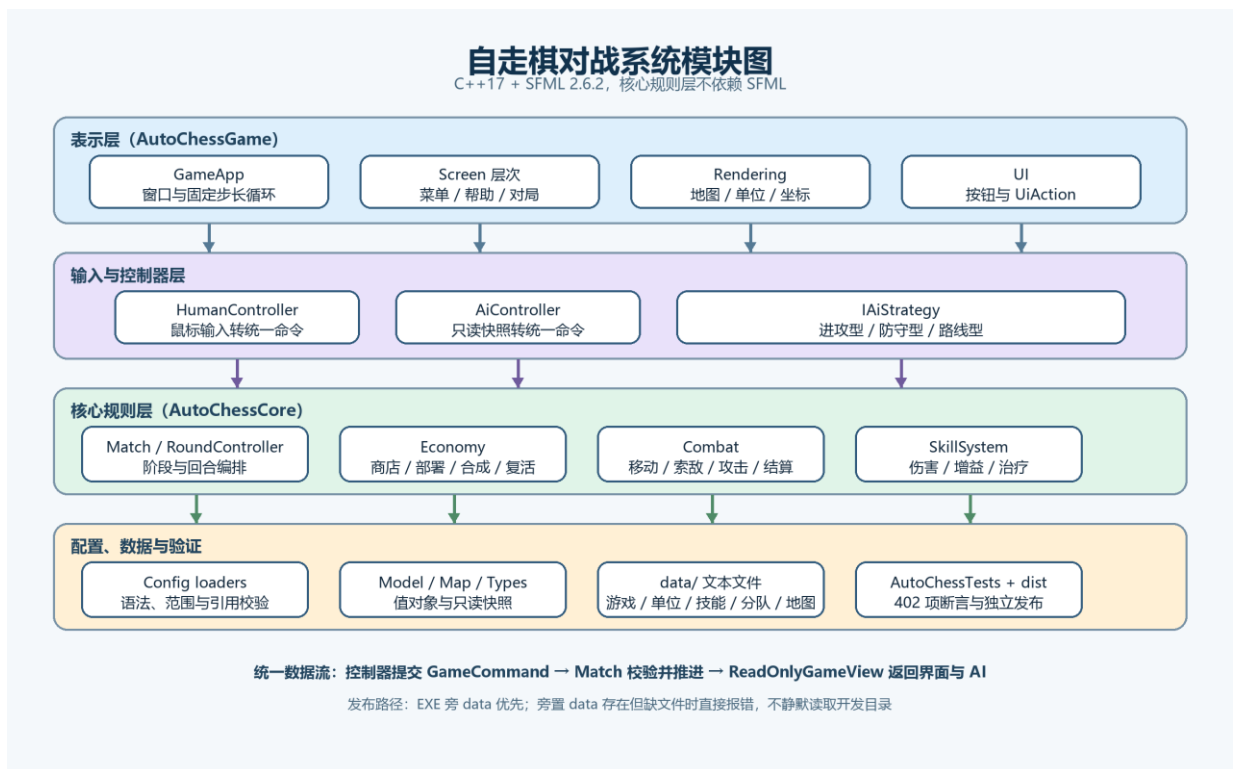


图 2-1 系统功能模块图

2.2 关键设计原则

原则	实现方式与收益
持久单位与战斗单位分离	OwnedUnit 保存跨回合状态，BattleUnit 只服务当前战斗，便于处理死亡、复活和回合恢复。
统一命令入口	HumanController 与 AiController 都提交 GameCommand，由 Match 负责阶段、归属和资源校验。
只读快照	ReadOnlyGameView 使用值拷贝，不向界面和 AI 暴露 Match 内部可写对象。
固定步长	准备与战斗均按 1/60 秒推进，渲染帧率变化不改变规则顺序。
数据驱动	单位、分队、经济和地图来自配置文件，界面不硬编码规则数值。
外部库隔离	Spine 单独编译为 AutoChessSpine，SFML 通过导入目标链接，依赖关系集中在 CMake 中。

2.3 模块依赖与数据流

1. GameApp 从 main.cpp 接收已选择的数据目录，加载 ConfigBundle、三套界面字体和动画资源路径。
2. Screen 将鼠标操作转换为 UiAction，HumanController 再把对局操作转换为 GameCommand。
3. AiController 读取 B 方 ReadOnlyGameView，并委托 IAIStrategy 生成至多一条购买或部署命令。
4. Match::submit 校验命令，Match::step 以 1/60 秒步长推进准备倒计时、BattleSimulation 和回合结算。
5. MatchScreen 根据只读快照绘制 HUD、地图和单位，并把战斗动作序号同步为 Spine 动画状态。
6. CMake 在链接后复制 DLL 与 data；程序运行时优先从 EXE 同级目录读取配置、字体、地图和动画文件。

3 系统详细设计

3.1 数据文件设计

核心配置使用 UTF-8 文本。加载顺序为 game.cfg、units.cfg、factions.cfg，再依次读取 map_01.map 至 map_04.map。解析器检查重复节、重复字段、未知字段、非法整数或比例、缺失字段、范围越界和跨文件引用。字体位于 data/fonts，五种单位各有准备区和战斗区两套 Spine 资源，每套由 atlas、png、skel 三个文件组成。

文件或目录	主要内容	当前规模
game.cfg	回合、准备时间、战斗超时、经济、容量、比例和随机种子	3 回合；45 秒准备；60 秒战斗；种子 20260814
units.cfg	生命、攻防、射程、速度、价格、守卫伤害和普通行为	铁卫、决斗者、游侠、奥术师、医师
factions.cfg	守卫、部署上限、价格倍率和单位属性修正	坚壁、突击、行军 3 个分队
map_01~map_02	双路训练场与多路线峡谷	11×7 双路；15×9 三路
map_03~map_04	中心对称三路要塞与镜像绕行回廊	19×11 三路；8×5 绕行路线
fonts 与 animations	中文字体和五种单位的准备/战斗 Spine 资源	1 个字体；30 个动画文件

3.2 类层次、抽象类与多态

Unit、Screen、IAIStrategy 和 IPlayerController 都是抽象层。IronGuardUnit、DuelistUnit、RangerUnit、ArcanistUnit、MedicUnit 通过重写 role() 表现运行时多态；MainMenuScreen、HelpScreen 和 MatchScreen 通过 Screen 接口统一处理事件与绘制；三种 AI 策略通过 IAIStrategy 统一选择命令。当前代码在删除主动技能模块后没有保留多重继承，这是与任务书形式要求尚未完全一致的一点，报告中不虚构已经不存在的类。

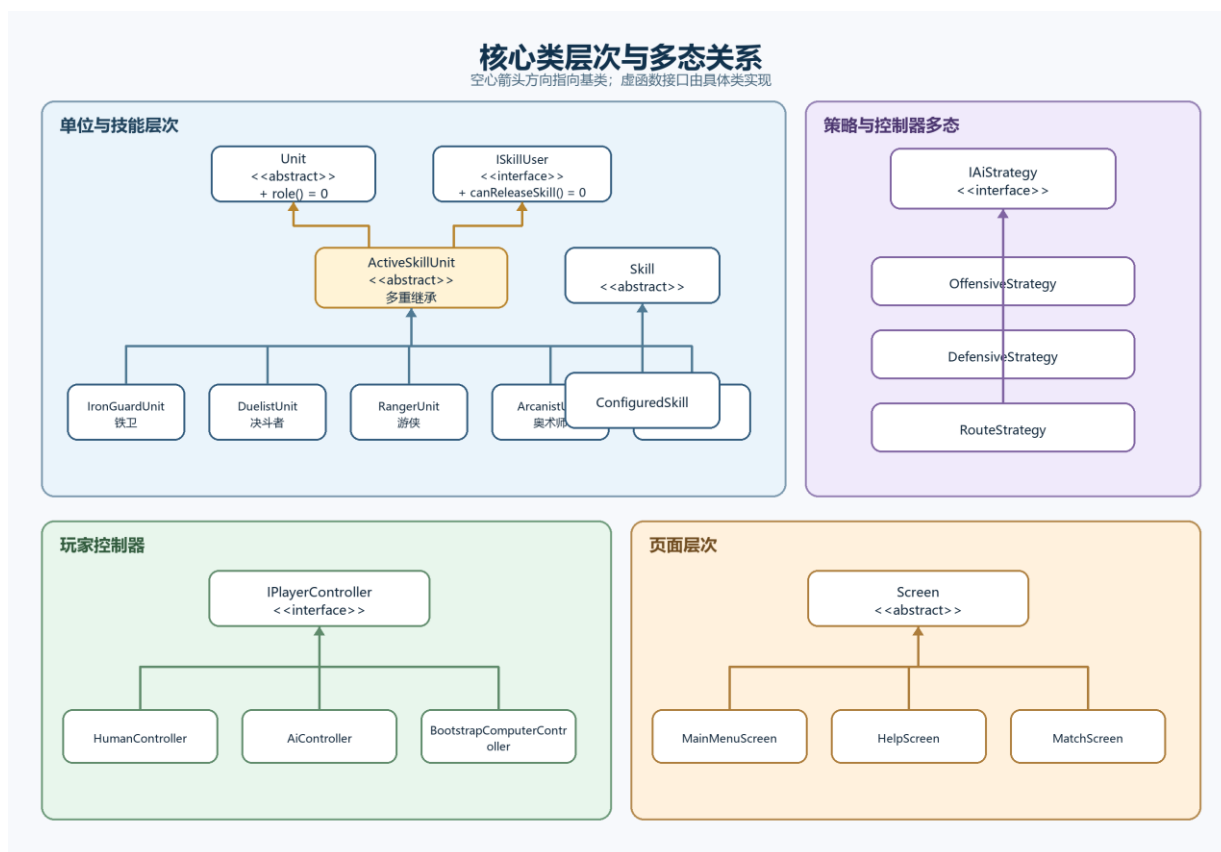


图 3-1 核心类层次与多态关系

代表性类	主要属性（均多于 3 项）	主要方法（均多于 3 项）
GameApp	dataDirectory_、window_、logicalView_、三套字体、config_、match_、双方控制器、screen_、frameClock_ 等	run、loadConfiguration、loadFonts、handleEvent、processUiAction、startNewGame、fixedUpdate、render 等
Match	config_、randomEngine_、phase_、currentRound_、地图/分队/AI 选择、双方玩家与商店、battle_ 等	submit、step、viewFor、selectMap、selectFaction、selectAiStrategy、startCombat、settleCurrentRound 等
BattleSimulation	units_、map_、timeoutFrames_、currentFrame_、finished_、守卫伤害和 summary_	step、moveUnit、updateTargets、applyBasicActions、applyFrameGuardDamage、finish 等
MatchScreen	只读快照、选择项、商店卡、复活卡、拖拽状态、按钮、HUD、动画仓库和动画实例表	handleEvent、draw、updateView、updateAnimations、rebuildChoices、finishBasicDrag、drawCombatUnits、syncAnimations 等

3.3 运算符重载

为简化值对象比较与日志输出，GridPosition 重载 ==、!=、<，BattlePosition、UnitIdentity 和 PlayerState 中使用的标识对象重载 ==，RoundSummary 与 MatchResultSummary 重载流输出 operator<<。MatchScreen 内部的 AnimationUnitKey 还重载 ==，用于 unordered_map 的动画实例索引。

3.4 经济与准备阶段

每回合开始按配置发放 5 金币，败者额外获得 2 金币。商店有 6 个槽位，刷新费为 2；备用区容量为 8。购买、刷新、部署、回备用区、合成、出售和复活均先验证再一次性修改状态。两个同类型同等级单位可合成，最高 3 级；合成返还 40%，出售返还 75%，复活成本为原价的 50%，统一使用冻结的四舍五入规则。

3.5 战斗、技能与结算

BattleSimulation 每帧先检查超时，再清理无效目标；没有目标的单位沿路线移动，随后统一处理守卫伤害、重新索敌和普通行动。攻击或治疗先形成同帧意图，再集中生效，避免遍历顺序改变结果。铁卫、决斗者、游侠和奥术师执行物理或法术攻击，医师的普通行为是治疗受伤友军。界面根据单位移动、攻击和死亡状态播放对应 Spine 动作，但动画只表现结果，不反向修改核心规则。

回合结束后，RoundController 把战斗结果按单位 ID 回写：死亡进入死亡列表，存活、到达守卫和超时单位回到活动名单并恢复下一回合状态。守卫归零立即判负；完成第 3 回合后比较守卫，数值相同则平局。

3.6 AI 设计

策略	决策偏好	可观察差异
进攻型 Offensive	优先高攻击、攻速与守卫伤害单位，并优先较短路线。	更快形成进攻压力。
防守型 Defensive	优先生命、防御、法抗和治疗单位，并针对主要威胁路线部署。	阵容更重视生存与保护。
路线型 Route	综合移动时间、射程、守卫伤害和价格评价单位与路线。	更重视路线效率。

3.7 界面设计

SFML 创建 2560×1440 物理窗口，并用 1280×720 逻辑视图保持界面比例。BoardTransform 负责格子和像素坐标换算，MapRenderer 绘制障碍、部署区、守卫和路线，UnitRenderer 提供静态回退图形，SpineAssetRepository 缓存 atlas、纹理和骨骼数据，UnitAnimationInstance 为每个单位维护独立动作状态。MatchScreen 只依据 ReadOnlyGameView 更新控件和动画，不直接写入 Match。

4 系统实现

4.1 固定步长主循环

GameApp 把真实帧间隔限制在最大值内，再用累加器按 FixedStepSeconds 重复调用 fixedUpdate。渲染频率和模拟步长分离后，暂停、测试和不同机器帧率不会改变规则顺序。

```
// 主循环只收集窗口事件，规则更新固定为每秒 60 次。
while (window_.isOpen())
{
    sf::Event event{};
    while (window_.pollEvent(event))
    {
        handleEvent(event);
    }

    const float frameSeconds = std::min(
        frameClock_.restart().asSeconds(), MaximumFrameSeconds);
    accumulatorSeconds_ += frameSeconds;
    while (accumulatorSeconds_ >= FixedStepSeconds)
    {
        fixedUpdate();
        accumulatorSeconds_ -= FixedStepSeconds;
    }
    render();
}
```

4.2 抽象类、继承与单位角色

```
// Unit 是抽象基类，五种具体单位通过虚函数报告角色。
class Unit
{
public:
    virtual ~Unit() = default;
    const UnitDefinition& definition() const noexcept;
    virtual UnitRole role() const noexcept = 0;
};

class IronGuardUnit final : public Unit
{
public:
    explicit IronGuardUnit(UnitDefinition definition);
    UnitRole role() const noexcept override;
};

// 真人与 AI 控制器都通过同一接口读取快照并返回命令。
class IPlayerController
{
public:
    virtual ~IPlayerController() = default;
    virtual MapSide side() const noexcept = 0;
    virtual std::optional<GameCommand> nextCommand(
        const ReadOnlyGameView& view) = 0;
};
```

4.3 统一命令

```
// variant 限定所有可以提交给 Match 的命令类型。
using GameCommandPayload = std::variant<
    SelectMapCommand, SelectFactionCommand, SelectAiStrategyCommand,
    RefreshShopCommand, PurchaseUnitCommand,
    MoveToDeploymentCommand, MoveToReserveCommand,
    MergeUnitsCommand, SellUnitCommand, ReviveUnitCommand,
```

```

    StartCombatCommand>;

struct GameCommand
{
    MapSide actor = MapSide::Unknown;
    GameCommandPayload payload = RefreshShopCommand{};
};

```

4.4 独立发布路径

程序不能依赖启动时的当前工作目录。入口通过 `GetModuleFileNameW` 得到 EXE 目录；只要旁边存在 `data` 目录，就固定使用该目录并暴露缺文件错误。这样既支持开发构建目录回退，也防止发布包缺文件时静默读取源码数据。

```

// 此函数让发布包使用自身 data，同时保留构建目录缺 data 时的开发回退。
std::filesystem::path selectDataDirectory(
    const std::filesystem::path& executableDirectoryPath,
    const std::filesystem::path& developmentDataDirectory)
{
    const std::filesystem::path adjacentDataDirectory =
        executableDirectoryPath / "data";
    std::error_code error;

    // 此代码块只要确认程序旁存在目录，就固定使用该目录并暴露其中缺文件错误。
    if (std::filesystem::is_directory(adjacentDataDirectory, error))
    {
        return adjacentDataDirectory;
    }
    return developmentDataDirectory;
}

```

5 系统测试

5.1 测试环境与方法

项目	当前环境或方法
操作系统与架构	Windows x64
语言与图形库	C++17, SFML 2.6.2
动画运行库	Spine Runtime 3.8, 项目内编译 spine-cpp 与 spine-sfml
编译器	MSVC 19.51.36256 x64
构建工具	CMake 3.29.2, Ninja 1.12.0, msvc-release 预设
构建目标	AutoChessCore、AutoChessSpine、AutoChessGame
验证方式	Release 构建、输出清单核对、配置检查和主要界面流程截图

5.2 编译与资源复制结果

测试范围	结果	依据
Release x64 构建	通过	生成 AutoChessGame.exe、AutoChessCore.lib、AutoChessSpine.lib
运行依赖复制	通过	3 个 SFML DLL、3 个 MSVC DLL 与 data 位于 EXE 同级
配置文件加载	通过	3 个 cfg 文件和 4 张地图均存在于构建输出
动画资源完整性	通过	5 种单位 × 2 套 × atlas/png/skel, 共 30 个文件
主要界面流程	通过	主菜单、帮助、选择、准备、战斗、暂停与结果截图

自动化回归	未建立	当前 CMakeLists.txt 中没有测试可执行目标
-------	-----	------------------------------

5.3 功能与异常场景验证

问题现象	排查方法	处理结果
找不到 SFML 头文件	检查 include 搜索路径是否指向 third_party/SFML/include	由导入目标统一提供头文件目录
SFML 符号无法链接	确认 x64 Release 的 graphics/window/system.lib 均已加入	三个导入目标通过 target_link_libraries 链接
双击程序提示缺 DLL	区分 .lib 与 .dll 的作用并检查 EXE 同级文件	构建后自动复制 3 个 SFML DLL 和 3 个 MSVC DLL
Spine 类型能包含但无实现	确认 spine-cpp 源文件和 spine-sfml.cpp 是否编入目标	建立独立 AutoChessSpine 静态库
动画文件读取失败	核对 atlas、png、skel 的目录、文件名和版本	资源集中放入 data/animations/units
换工作目录后找不到 data	比较当前目录与 EXE 目录	优先读取 EXE 同级 data
中文字体缺失或模糊	检查系统字体和高分辨率缩放	多字体回退并采用 1280×720 逻辑视图
死亡动画位置偏移	比较骨骼可见边界、缩放高度和动作锚点	按动作计算比例并对铁卫死亡动作校准

5.4 测试结论与不足

当前副本没有 AutoChessTests 自动测试目标，因此本报告不再沿用旧稿中的 402 项断言数据。本次以 Release x64 重新构建结果、生成目录文件清单、关键配置检查和现有界面操作截图作为验证依据。build/msvc-release 中已生成 AutoChessGame.exe、AutoChessCore.lib、AutoChessSpine.lib、3 个 SFML DLL、3 个 MSVC 运行库和完整 data 目录。data 共包含 3 个 cfg 文件、4 张地图、1 个中文字体文件以及 30 个 Spine 动画文件。构建与资源复制通过；自动化回归覆盖不足仍需后续补充。

6 运行界面与结果

以下截图记录了主菜单、帮助、选择、准备、战斗、暂停和继续等主要交互。截图用于说明界面流程；当前版本已在同一界面结构上加入 Spine 角色动画，并删除主动技能按钮。



图 6-1 主菜单：开始游戏、帮助与退出

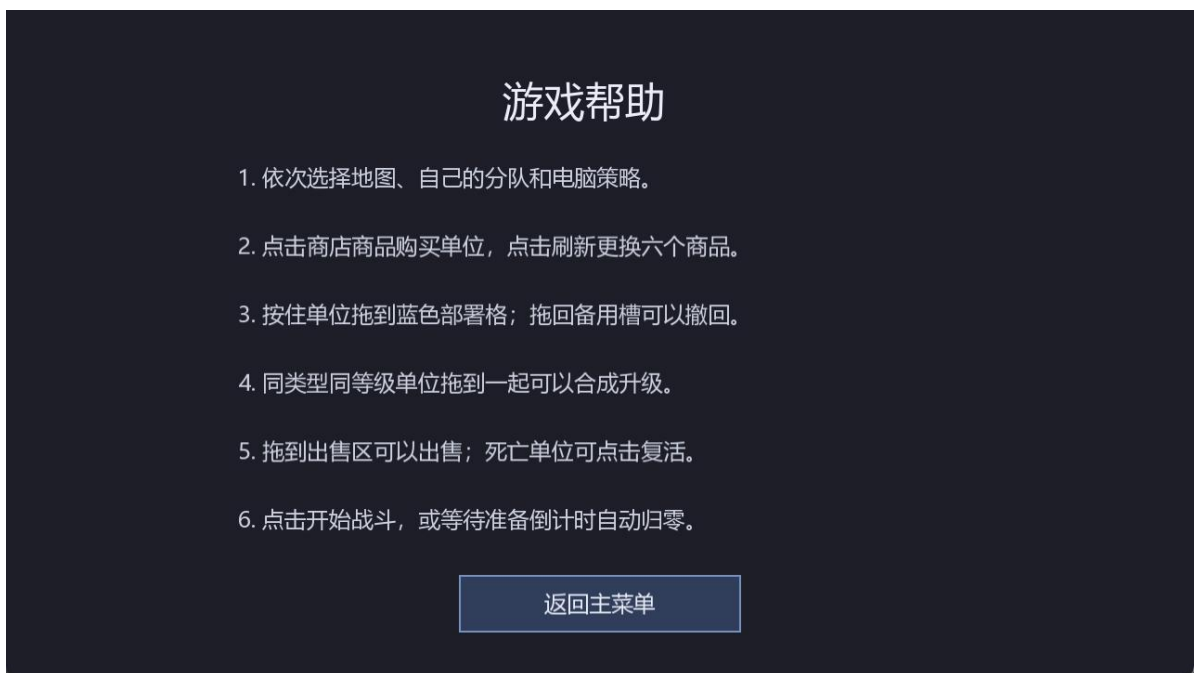


图 6-2 帮助页：新手操作说明



图 6-3 地图选择



图 6-4 玩家分队选择



图 6-5 AI 策略选择

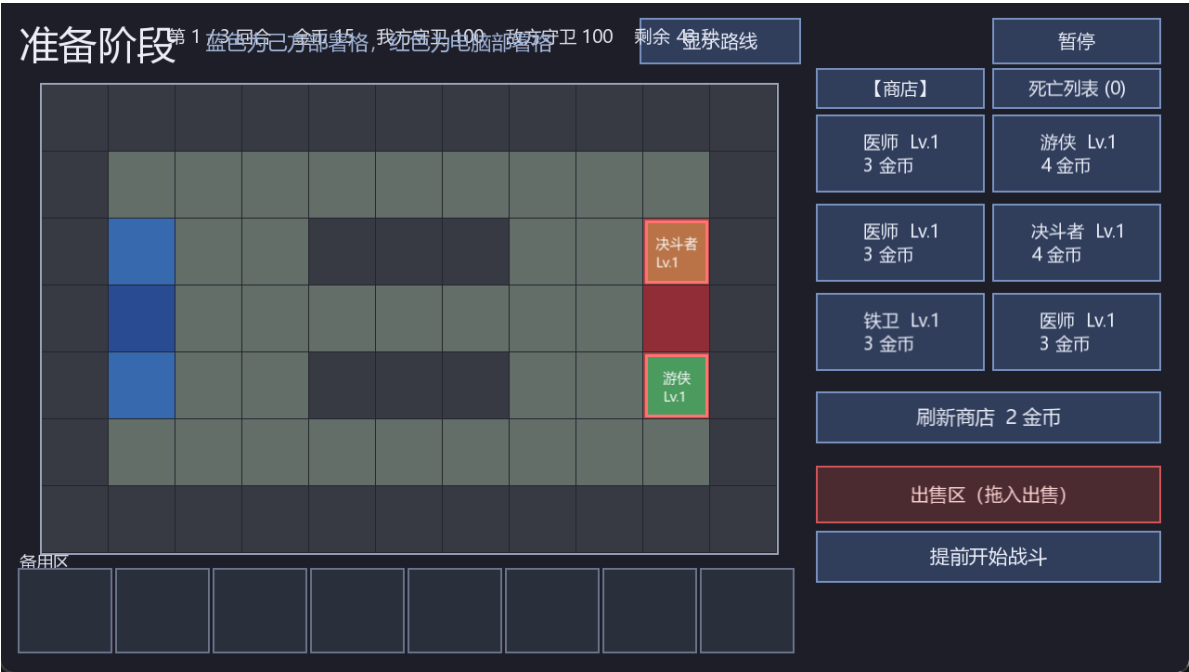


图 6-6 准备阶段完整 HUD 与商店



图 6-7 购买单位后备用区状态



图 6-8 拖拽部署后的棋盘状态

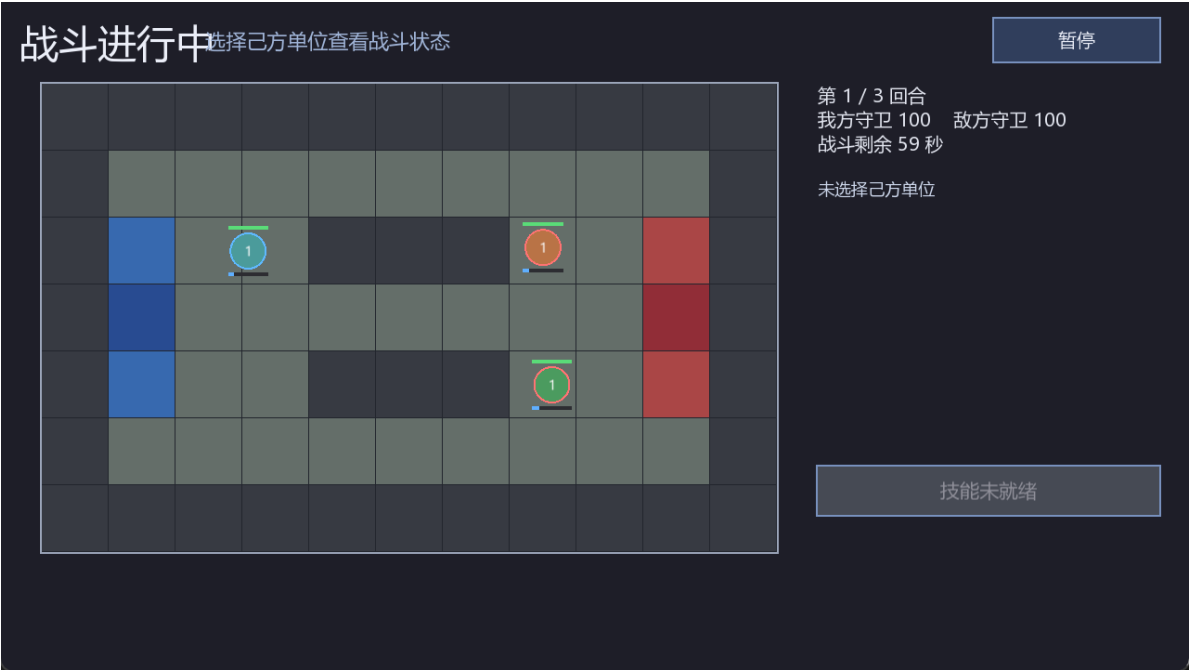


图 6-9 战斗单位、生命状态与目标连线



图 6-10 暂停覆盖层



图 6-11 继续后恢复战斗推进

7 系统调试与遇到的困难

7.1 从单文件程序到 CMake 多文件项目

在此之前我只学习过 C 和 C++，练习通常是一个或少量源文件，没有接触过完整工程。开始时我不理解头文件、cpp 文件、静态库和可执行文件为什么要分成不同目标，也分不清“编译错误”和“链接错误”。解决方法是先画出 AutoChessCore、AutoChessSpine、AutoChessGame 三者关系，再在 CMakeLists.txt 中逐个列出源文件、头文件搜索路径和 target_link_libraries。当某个符号无法找到时，我先确认声明是否可见，再确认实现文件是否加入目标，最后确认目标之间是否链接。这样把一个模糊的“工程不能运行”拆成配置、编译、链接和运行四个阶段。

7.2 SFML 外部库的编译、链接与运行

SFML 是我第一次真正接入的外部库。最容易混淆的是三个层次：include 目录只让编译器找到 .hpp；.lib 文件让链接器找到函数实现；.dll 文件必须在程序运行时位于 EXE 可搜索的位置。只完成其中一层时，会分别出现找不到头文件、链接符号失败或双击程序后提示缺少 DLL。项目最后在 CMake 中把 system、window、graphics 声明为三个 IMPORTED 目标，统一指定 x64 Release 导入库、DLL 和头文件目录，并用 POST_BUILD 命令把三个 DLL 复制到 AutoChessGame.exe 旁。这一过程让我第一次理解“写 C++ 代码”和“把第三方代码接进项目”是两件不同的工作。

7.3 Spine 动画库与资源文件接入

Spine 的接入比 SFML 更复杂，因为它不仅需要头文件和库，还要编译 spine-cpp 的大量源文件，再与 spine-sfml 适配层和 SFML 链接。开始时我不知道应该把外部源码单独做成库，若直接散放进游戏目标，很难判断错误来自自己的代码还是运行库。最终建立 AutoChessSpine 静态库，集中设置 include 路径、编译警告和依赖关系。运行时还必须保证每套动画的 atlas、png、skel 版本一致、文件名一致。SpineAssetRepository 负责加载和缓存，失败时返回错误并使用静态图形回退，从而避免一个角色资源缺失导致整个程序崩溃。

7.4 路径、字体和动画坐标调试

- 资源路径：程序从其他目录启动时找不到 data。使用 GetModuleFileNameW 获取 EXE 目录，并优先读取同级 data。
- 中文字体：不同电脑不一定有同一种字体。程序依次尝试宋体、华文宋体和随项目附带的 Noto Sans SC。
- 动画比例：Spine 原始骨骼尺寸与棋盘格不一致。按骨骼可见边界计算 scaleForHeight，再针对铁卫死亡动作校准偏移。
- 调试方法：先保存完整错误信息，判断属于配置、编译、链接还是运行；一次只改一个原因，重新构建后再验证。

8 使用说明与编程体会

8.1 发布程序使用步骤

7. 打开 build/msvc-release 目录，确认 AutoChessGame.exe、6 个运行时 DLL 和 data 目录位于同一级。
8. 双击 AutoChessGame.exe；资源完整时将出现标题为 AutoChess 的主菜单窗口。
9. 首次使用可点击“帮助”，阅读购买、拖拽、合成、出售、复活和开始战斗的方法。
10. 点击“开始游戏”，依次选择四张地图之一、玩家分队和 AI 策略。
11. 准备阶段点击右侧单位卡购买；按住己方单位拖到蓝色部署格，或拖回备用区和出售区。
12. 需要时点击刷新、合成或复活；电脑完成部署后可提前开战，也可等待 45 秒倒计时结束。
13. 战斗会自动移动、索敌、攻击或治疗，并播放 relax、move、attack、die 动画；可使用暂停和继续。
14. 每回合结算后自动进入下一回合，最多 3 回合；结果页可再来一局或返回主菜单。

8.2 构建与运行步骤

15. 打开 Visual Studio x64 Developer Command Prompt，并切换到项目根目录。
16. 执行 `cmake --fresh --preset msvc-release`，生成 Ninja Release 构建文件。
17. 执行 `cmake --build --preset msvc-release --parallel`，生成 AutoChessCore、AutoChessSpine 和 AutoChessGame。
18. 进入 `build/msvc-release`，检查 DLL 和 data 已由构建脚本自动复制，然后运行 AutoChessGame.exe。
19. 移动程序时应整体复制 AutoChessGame.exe、6 个运行时 DLL 和 data 目录，不能只复制 EXE。

8.3 编程体会

我此前只学过 C 和 C++，没有学过 CMake、图形库、骨骼动画库，也没有真正处理过项目之间的依赖。这次最重要的收获不是又写了几个类，而是第一次看懂一个程序从源文件到可执行文件的完整过程：预处理阶段寻找头文件，编译阶段把 cpp 变成目标文件，链接阶段把自己的目标文件与 .lib 组合，运行阶段再由系统加载 DLL 和数据资源。理解这四步以后，面对大量报错时不再只是反复修改代码。

另一个收获是学会控制项目边界。核心规则放在不依赖 SFML 的 AutoChessCore 中，外部 Spine 源码放在 AutoChessSpine 中，界面代码只负责输入与显示；运行资源统一复制到 EXE 旁。这种组织方式刚开始比写单文件程序麻烦，但后期定位配置、链接、路径和动画问题更清楚。当前项目仍缺少独立自动测试，也没有保留任务书要求的多重继承，这两点是后续首先需要补齐的内容。

9 总结与展望

本项目已完成自走棋的地图与分队选择、商店经济、购买与刷新、拖拽部署、合成、出售、复活、AI 准备、固定步长战斗、回合结算、暂停和结果页面，并成功接入 SFML 与 Spine 3.8。代码具有抽象类、单继承、多态、运算符重载、文件读写和明确的模块边界，Release x64 构建可以生成完整运行目录。项目也暴露出两个明确不足：当前副本缺少自动测试目标；删除技能系统后不再有多重继承。

后续应先补充不依赖窗口的核心规则测试，并根据课程要求设计一个具有真实职责的第二接口，让合适的派生类采用多重继承，而不是为了形式随意增加空基类。之后可继续增加音效、更多地图、动画混合、动态寻路或网络控制器。扩展时仍应保持 GameCommand 和 ReadOnlyGameView 边界，避免界面或动画直接修改核心状态。

附录 A 关键源程序与源程序清单

第 4 章列出了固定步长循环、抽象类、统一命令和运行路径等关键代码并附用途注释。下表列出当前 src 目录中的 94 个 C++ 源文件与头文件，不包含已经从当前副本删除的测试和技能模块。

序号	文件	职责
1	src/core/Core.cpp	核心公共入口
2	src/core/Core.hpp	核心公共入口
3	src/core/ai/AiController.cpp	AI 接口、决策工具、策略与控制器
4	src/core/ai/AiController.hpp	AI 接口、决策工具、策略与控制器
5	src/core/ai/AiDecisionSupport.cpp	AI 接口、决策工具、策略与控制器
6	src/core/ai/AiDecisionSupport.hpp	AI 接口、决策工具、策略与控制器
7	src/core/ai/DefensiveStrategy.cpp	AI 接口、决策工具、策略与控制器
8	src/core/ai/DefensiveStrategy.hpp	AI 接口、决策工具、策略与控制器
9	src/core/ai/IAiStrategy.hpp	AI 接口、决策工具、策略与控制器
10	src/core/ai/OffensiveStrategy.cpp	AI 接口、决策工具、策略与控制器
11	src/core/ai/OffensiveStrategy.hpp	AI 接口、决策工具、策略与控制器
12	src/core/ai/RouteStrategy.cpp	AI 接口、决策工具、策略与控制器
13	src/core/ai/RouteStrategy.hpp	AI 接口、决策工具、策略与控制器
14	src/core/combat/BattleSetupService.cpp	固定步长战斗、属性结算、索敌与战斗数据
15	src/core/combat/BattleSetupService.hpp	固定步长战斗、属性结算、索敌与战斗数据
16	src/core/combat/BattleSimulation.cpp	固定步长战斗、属性结算、索敌与战斗数据
17	src/core/combat/BattleSimulation.hpp	固定步长战斗、属性结算、索敌与战斗数据
18	src/core/combat/BattleStatResolver.cpp	固定步长战斗、属性结算、索敌与战斗数据
19	src/core/combat/BattleStatResolver.hpp	固定步长战斗、属性结算、索敌与战斗数据
20	src/core/combat/BattleTypes.hpp	固定步长战斗、属性结算、索敌与战斗数据
21	src/core/combat/CombatRules.cpp	固定步长战斗、属性结算、索敌与战斗数据
22	src/core/combat/CombatRules.hpp	固定步长战斗、属性结算、索敌与战斗数据
23	src/core/combat/TargetSelector.cpp	固定步长战斗、属性结算、索敌与战斗数据
24	src/core/combat/TargetSelector.hpp	固定步长战斗、属性结算、索敌与战斗数据
25	src/core/config/ConfigBundleLoader.cpp	配置解析、类型读取、文件加载与校验
26	src/core/config/ConfigBundleLoader.hpp	配置解析、类型读取、文件加载与校验
27	src/core/config/ConfigError.hpp	配置解析、类型读取、文件加载与校验
28	src/core/config/ConfigParser.cpp	配置解析、类型读取、文件加载与校验
29	src/core/config/ConfigParser.hpp	配置解析、类型读取、文件加载与校验
30	src/core/config/ConfigValueReader.cpp	配置解析、类型读取、文件加载与校验
31	src/core/config/ConfigValueReader.hpp	配置解析、类型读取、文件加载与校验
32	src/core/config/DefinitionConfigLoader.cpp	配置解析、类型读取、文件加载与校验
33	src/core/config/DefinitionConfigLoader.hpp	配置解析、类型读取、文件加载与校验
34	src/core/config/GameConfigLoader.cpp	配置解析、类型读取、文件加载与校验
35	src/core/config/GameConfigLoader.hpp	配置解析、类型读取、文件加载与校验
36	src/core/config/MapConfigLoader.cpp	配置解析、类型读取、文件加载与校验
37	src/core/config/MapConfigLoader.hpp	配置解析、类型读取、文件加载与校验
38	src/core/controllers/IPlayerController.hpp	玩家控制器抽象接口
39	src/core/economy/DeploymentService.cpp	商店、部署、合成、出售、复活与价格规则
40	src/core/economy/DeploymentService.hpp	商店、部署、合成、出售、复活与价格规则
41	src/core/economy/EconomyTypes.hpp	商店、部署、合成、出售、复活与价格规则
42	src/core/economy/MergeService.cpp	商店、部署、合成、出售、复活与价格规则
43	src/core/economy/MergeService.hpp	商店、部署、合成、出售、复活与价格规则
44	src/core/economy/PriceRules.cpp	商店、部署、合成、出售、复活与价格规则
45	src/core/economy/PriceRules.hpp	商店、部署、合成、出售、复活与价格规则
46	src/core/economy/RosterService.cpp	商店、部署、合成、出售、复活与价格规则
47	src/core/economy/RosterService.hpp	商店、部署、合成、出售、复活与价格规则
48	src/core/economy/ShopService.cpp	商店、部署、合成、出售、复活与价格规则
49	src/core/economy/ShopService.hpp	商店、部署、合成、出售、复活与价格规则
50	src/core/map/MapTypes.hpp	地图、路线与格子数据
51	src/core/match/GameCommand.hpp	统一命令、对局阶段、回合结算与只读视图
52	src/core/match/Match.cpp	统一命令、对局阶段、回合结算与只读视图
53	src/core/match/Match.hpp	统一命令、对局阶段、回合结算与只读视图
54	src/core/match/MatchTypes.cpp	统一命令、对局阶段、回合结算与只读视图
55	src/core/match/MatchTypes.hpp	统一命令、对局阶段、回合结算与只读视图
56	src/core/match/ReadOnlyGameView.hpp	统一命令、对局阶段、回合结算与只读视图
57	src/core/match/RoundController.cpp	统一命令、对局阶段、回合结算与只读视图
58	src/core/match/RoundController.hpp	统一命令、对局阶段、回合结算与只读视图
59	src/core/model/Definitions.hpp	单位、玩家、分队与配置基础模型
60	src/core/model/PlayerStateService.cpp	单位、玩家、分队与配置基础模型
61	src/core/model/PlayerStateService.hpp	单位、玩家、分队与配置基础模型
62	src/core/model/PlayerTypes.hpp	单位、玩家、分队与配置基础模型
63	src/core/units/Unit.cpp	单位抽象类、具体单位与角色多态

自走棋对战系统课程设计报告

64	src/core/units/Unit.hpp	单位抽象类、具体单位与角色多态
65	src/game/GameApp.cpp	应用入口、窗口主循环与页面编排
66	src/game/GameApp.hpp	应用入口、窗口主循环与页面编排
67	src/game/RuntimePaths.cpp	可执行文件目录与运行资源路径
68	src/game/RuntimePaths.hpp	可执行文件目录与运行资源路径
69	src/game/animation/SpineAssetRepository.cpp	Spine 资源加载、缓存、动作状态与绘制
70	src/game/animation/SpineAssetRepository.hpp	Spine 资源加载、缓存、动作状态与绘制
71	src/game/animation/UnitAnimationInstance.cpp	Spine 资源加载、缓存、动作状态与绘制
72	src/game/animation/UnitAnimationInstance.hpp	Spine 资源加载、缓存、动作状态与绘制
73	src/game/animation/UnitAnimationManifest.cpp	Spine 资源加载、缓存、动作状态与绘制
74	src/game/animation/UnitAnimationManifest.hpp	Spine 资源加载、缓存、动作状态与绘制
75	src/game/controllers/HumanController.cpp	真人输入到统一命令的转换
76	src/game/controllers/HumanController.hpp	真人输入到统一命令的转换
77	src/game/main.cpp	应用入口、窗口主循环与页面编排
78	src/game/rendering/BoardTransform.cpp	坐标换算、地图与单位静态绘制
79	src/game/rendering/BoardTransform.hpp	坐标换算、地图与单位静态绘制
80	src/game/rendering/MapRenderer.cpp	坐标换算、地图与单位静态绘制
81	src/game/rendering/MapRenderer.hpp	坐标换算、地图与单位静态绘制
82	src/game/rendering/UnitRenderer.cpp	坐标换算、地图与单位静态绘制
83	src/game/rendering/UnitRenderer.hpp	坐标换算、地图与单位静态绘制
84	src/game/screens/HelpScreen.cpp	菜单、帮助、对局页面与界面流程
85	src/game/screens/HelpScreen.hpp	菜单、帮助、对局页面与界面流程
86	src/game/screens/MainMenuScreen.cpp	菜单、帮助、对局页面与界面流程
87	src/game/screens/MainMenuScreen.hpp	菜单、帮助、对局页面与界面流程
88	src/game/screens/MatchScreen.cpp	菜单、帮助、对局页面与界面流程
89	src/game/screens/MatchScreen.hpp	菜单、帮助、对局页面与界面流程
90	src/game/screens/Screen.hpp	菜单、帮助、对局页面与界面流程
91	src/game/ui/Button.cpp	按钮、主题与界面动作
92	src/game/ui/Button.hpp	按钮、主题与界面动作
93	src/game/ui/UiAction.hpp	按钮、主题与界面动作
94	src/game/ui/UiTheme.hpp	按钮、主题与界面动作

附录 B 完整课程设计任务书与评分表

以下 4 页保留课程设计任务书与评分表，放在源程序清单之后，便于教师核对课程要求与评分项目。

课程设计报告

课程名称 计算机程序设计基础2

班 级 _____
学 号 _____
姓 名 _____

2026 年 5 月 19 日

附图 B-1 原课程设计任务书第 1 页

一、设计内容与设计要求

1. 课程设计目的

面向对象程序设计课程是集中实践性环节之一，是学习完《计算机程序设计基础2》C++面向对象程序设计课程后进行的一次全面的综合练习。要求学生达到熟练掌握C++语言的基本知识和技能；基本掌握面向对象程序设计的思想和方法；能够利用所学的基本知识和技能，解决简单的面向对象程序设计问题，从而提高动手编程解决实际问题的能力。**尤其重视创新思维培养。**

2. 课题题目

1) 自走棋对战系统：自走棋是一种策略游戏，玩家需要购买、升级、排列棋子以击败对手。

3. 课题内容（仅供参考）

- 系统需求分析：分析自走棋对战系统的功能需求，包括棋子购买、升级、排列、对战等。
- 系统设计：设计自走棋对战系统的类层次结构，确定基类和派生类，以及类的成员函数和成员变量。
- 系统实现：
 - 1) 实现棋子类，包括棋子的名称、攻击力、血量等属性，以及攻击、移动等行为。
 - 2) 实现玩家类，包括玩家的名称、血量、金币、棋子等属性，以及购买、升级、排列棋子等行为。
 - 3) 实现战场类，包括战场的地图、棋子位置等属性，以及棋子的移动、攻击等行为。
 - 4) 实现系统的菜单功能，包括开始游戏、购买棋子、升级棋子、排列棋子、开始对战等。
- 设计策略型AI玩家互相对战。（加分项）

4. 文档设计要求

4.1 每个同学都单独完成1道课题。后面有范题，仅供同学们参考，不列入本次课程设计的课题。

4.2 对于程设题目，按照范题的格式。自行虚构软件需求。并按照第4点要求，编写设计文档。基本要求系统中设计的类的数目不少于4个，每个类中要有各自的属性（多于3个）和方法（多于3个）；需要定义一个抽象类，采用继承方式派生这些类。并设计一个多重继承的派生类。在程序设计中，引入虚函数的多态性、运算符重载等机制。

5. 程序设计的基本要求:

- (1) 要求利用面向对象的方法以及C++的编程思想来完成系统的设计;
- (2) 要求在设计的过程中, 建立清晰的类层次;
- (3) 根据课题完成以下主要工作: ①完成系统需求分析: 包括系统设计目的与意义; 系统功能需求(系统流程图); 输入输出的要求。②完成系统总体设计: 包括系统功能分析; 系统功能模块划分与设计(系统功能模块图)。③完成系统详细设计: 数据文件; 类层次图; 界面设计与各功能模块实现。④系统调试: 调试出现的主要问题, 编译语法错误及修改, 重点是运行逻辑问题修改和调整。⑤使用说明书及编程体会: 说明如何使用你编写的程序, 详细列出每一步的操作步骤。⑥关键源程序(带注释)。
- (4) 自己设计测试数据, 将测试数据存在文件中, 通过文件进行数据读写来获得测试结果。
- (5) 按规定格式完成课程设计报告, 并在网络学堂上按时提交。
- (6) 不得抄袭他人程序、课程设计报告, 每个人应独立完成, 在程序和设计报告中体现自己的个性设计。

5、进度安排

小学期 第3周			

注: 1、一定要保留自己那个课题的完整任务书在课程设计报告里面。

2、“评分表”放在“附录: 源程序清单”的后面。

附录1：评分表

项目	评价	
设计方案的合理性与创新性	3	
设计与调试结果	4	
设计说明书的质量	1	
程序基本要求涵盖情况	4	
程序代码编写素养情况	2	
课程设计周表现情况	1	
综合成绩	15	